Security & Compliance

Polycarpic — a virtual Single Family Office

Owner: `seceng` agent

Status: Draft v1 (reviewed rewrite, supersedes 8963efb). Pending the architect's re-fold (§12.2) and the Principal's review (§11 approvals, §12.3).

Traces to: PRD v1 (Approved 2026-09-27), ARCH v1 draft (@ db8aa61) **Last updated:** 2026-09-28

CONTENTS

1. Compliance Posture
2. Trust Boundaries
3. Threat Model (STRIDE)
4. Authentication & Authorization
5. Data Classification & Handling
6. Secret Management
7. Controls Catalog
8. CI/CD Security
9. Logging, Monitoring, Detection
10. Incident Response
11. Open Risks (Accepted)
12. Rulings on ARCH Open Questions

1. Compliance Posture

Source of truth is PRD §6 (Non-Functional Requirements) and the Principal's audience decision of 2026-09-27.

Polycarpic is software a person installs and runs for their own household. The Principal will never operate it as a service for others. That is a standing non-goal (PRD §3), not a v1 deferral. Nobody is anybody's "customer", so no formal compliance regime attaches.

That does not lower the bar. The Principal made the PRD §6 security items hard requirements, and this document treats them that way. Every control here answers a threat in §3, not an auditor. The yardstick is **OWASP ASVS 4.0 Level 2**, used as a checklist, not as a certification target.

Regime	In scope?	Why	Target evidence
GLBA Safeguards Rule	No	Applies to financial institutions that serve customers, and a self-hosted personal tool has none. It would likely attach if the hosting non-goal were reversed (R-6).	—
PCI-DSS	No	We never store, process or transmit card numbers. Aggregators expose transactions, not PANs. Adapters must not persist a PAN even if a provider payload carries one (C-34).	C-34 adapter fixture test
GDPR / CCPA	No (voluntary alignment)	Processing by a natural person in a purely personal or household activity falls under the GDPR Art. 2(2)(c) household exemption. CCPA thresholds are never met. We adopt the data-rights practices anyway (PRD §6): export, account deletion and entity purge, all scoped to the immutable ledger.	C-50, C-51, C-52
SOC 2	No	No service organization and no customers.	—
HIPAA	No	No health data by design. Payee strings can still reveal medical providers, which is one reason memo and payee text is Tier 3 (§5).	—
Aggregator terms (Plaid, SimpleFIN)	Yes, per operator	Each self-hoster obtains their own provider keys and accepts that provider's terms. Plaid production access comes with a security questionnaire. The controls here are written so an operator can answer it truthfully.	§6, C-30 to C-35

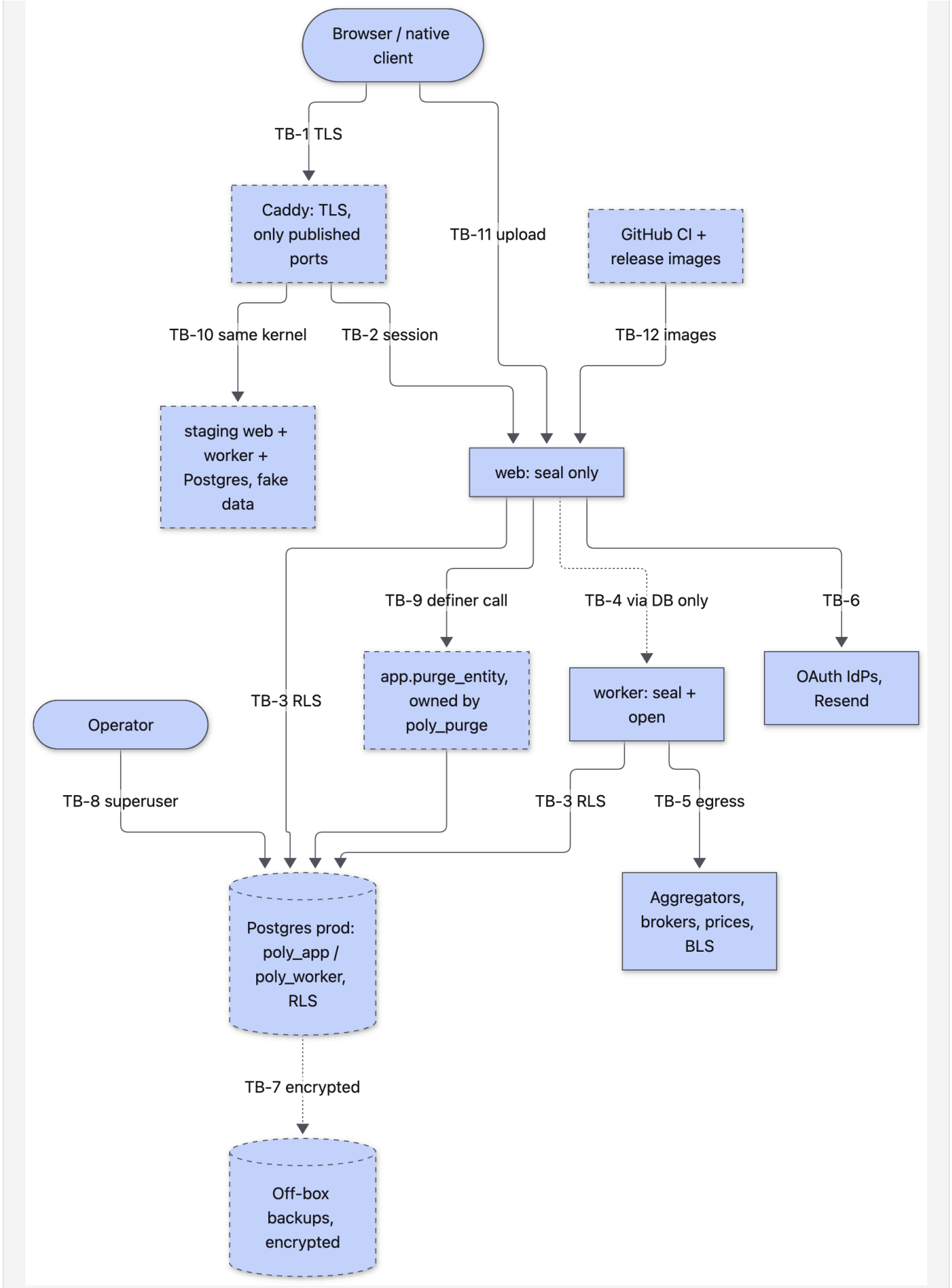
Regime	In scope?	Why	Target evidence
Tax record retention (IRS guidance, 3–7 years)	Advisory	Not a regime we enforce. The purge flow must warn the owner about it (PRD §5).	C-50 warning copy

2. Trust Boundaries

ARCH §7 cites TB-1 to TB-10, so those numbers stay stable. The OQ-1 revision (§6.3) redefines TB-4. TB-11 and TB-12 are new. Each boundary becomes a threat entry in §3.

#	From	To	Boundary type	Notes
TB-1	Internet	Caddy	Network, TLS	TLS 1.2+ with HSTS. Caddy is the only container that publishes ports (80 and 443).
TB-2	Browser, native and MCP clients	SvelteKit `web`	Authentication	A session cookie or bearer token goes through `getSession()` and becomes a branded `SessionUserId`. Anything else is anonymous. CSRF and origin checks sit here.
TB-3	`web` / `worker`	Postgres	DB role + RLS	The primary authorization boundary. Apps connect as `poly_app` or `poly_worker`, never as the owner. Tenant context is transaction-local (§4.3).
TB-4	`web`	`worker` (the credential-opening capability)	Process + key separation	`web` can seal credentials but cannot open them; only `worker` holds the private key (§6.3). The two share only the database. *Revised:* 8963efb had a symmetric master key mounted into the app.
TB-5	`worker`	Aggregators, brokers, price and CPI providers	Egress, third-party trust	Outbound only. Credentials are opened just in time. Every response is untrusted input: payee and memo text can be written by anyone who sends the user money.
TB-6	`web`	OAuth IdPs, Resend	Third-party identity and email	The IdP asserts identity. Resend sees verification and magic-link URLs in transit.
TB-7	Box	Off-box backup store	Custody	Backups are encrypted before egress, with a key separate from the credential key (§6.6).

#	From	To	Boundary type	Notes
TB-8	Operator (SSH, `docker compose exec`)	Box, Postgres superuser	Administrative	Bypasses RLS and triggers by definition (R-1). C-24 is the compensating control.
TB-9	Ordinary write path	Purge path	Privilege (DB role)	The only path that deletes posted rows. It runs inside a function owned by the NOLOGIN `poly_purge` role (§4.5).
TB-10	Staging compose project	Prod compose project	Container isolation, shared kernel	Staging auto-deploys every merge to `main` and holds fake data only. It has separate networks, secrets and database, and no credential private key (R-4).
TB-11	User upload (CSV / OFX)	`web` parser	Untrusted file	The credential-free import path, and a parser attack surface (C-40).
TB-12	GitHub (CI, Actions, Renovate, registry)	Release images, prod deploy	Supply chain	Whoever can publish an image that prod runs owns the box (§8).
TB-13	Operator (SSH tunnel)	Admin view listener, bound to loopback and never routed by Caddy	Administrative, out-of-band	*Placeholder; defined under POLY-81.* The design options are ARCH OQ-15: (a) a loopback route group inside `web`, or (b) a separate admin service with a read-only status DB role. The threats and controls are written when POLY-81 is ruled.


Browser / native client
TB-1 TLS
Caddy: TLS, only published ports
TB-11 upload
GitHub CI + release images
TB-10 same kernel
TB-2 session
TB-12 images
staging web + worker + Postgres, fake data
web: seal only
TB-9 definer call
TB-4 via DB only
TB-6
TB-3 RLS
app.purge_entity, owned by poly_purge
worker: seal + open
OAuth IdPs, Resend
Operator
TB-8 superuser
TB-3 RLS
TB-5 egress
Postgres prod: poly_app / poly_worker, RLS
Aggregators, brokers, prices, BLS
TB-7 encrypted
Off-box backups, encrypted

3. Threat Model (STRIDE)

One table per component group. Only threats with non-trivial likelihood × impact are listed. L = likelihood, I = impact (H/M/L). Control IDs refer to §7.

Attacker model.

- (a) An internet attacker who can reach Caddy.
- (b) A curious or malicious household member with a legitimate viewer or manager grant.
- (c) A counterparty who controls text that lands in our data. Anyone who sends the user money writes a payee or memo string.
- (d) A compromised dependency or CI path.
- (e) A thief with a backup or a disk.
- (f) An attacker with code execution in `web`. This one is in scope, and the design limits what they get.

The operator's own superuser is not modelled as an attacker (R-1).

3.1 TB-1 / TB-2 — Edge & web app (Caddy, SvelteKit, `packages/auth`)

ID	Threat	STRIDE	L	I	Control(s)
T-01	A stranger signs up on a public instance and gets an account on someone's household box.	S / E	H	M	C-01
T-02	Credential stuffing, spraying or brute force against password login.	S	M	H	C-03, C-04
T-03	OAuth account takeover: an IdP identity with an unverified email auto-links to an existing user.	S	L	H	C-05
T-04	Session cookie theft through XSS, subdomain cookie injection or the network.	S	M	H	C-02, C-07, C-41
T-05	CSRF on state-changing form actions or `/api/v1`.	T / E	M	H	C-06
T-06	Stored XSS through provider payee or memo strings, or through entity or report names set by a co-member.	T / E	M	H	C-41, C-07
T-07	Account enumeration through login, reset or signup responses.	I	M	L	C-04

ID	Threat	STRIDE	L	I	Control(s)
T-08	A stolen session is used for an irreversible or exfiltration-shaped action: purge, export, provider link, membership grant.	S / E	L	H	C-03 (step-up), C-50
T-09	Floods against expensive endpoints: reports, export, upload.	D	L	M	C-04, C-40, C-42

3.2 TB-3 — Database & tenancy (`packages/db` , `packages/tenancy` , RLS)

ID	Threat	STRIDE	L	I	Control(s)
T-10	IDOR: a route accepts an `entityId` the caller has no membership on.	I / T	H	H	C-13, C-14, C-17
T-11	A pooled connection carries tenant A's context into tenant B's request.	I	M	H	C-11, C-12
T-12	A new table ships without RLS: staging, draft, lot or some future table.	I	H	H	C-13 (catalog-derived battery)
T-13	RLS is bypassed through ownership: a view runs as its owner, a materialized view has no RLS, or a SECURITY DEFINER function is not on the allowlist.	I / E	M	H	C-15, C-16
T-14	A write policy without `WITH CHECK` , or an updatable `entity_id` , lets a row be inserted into or moved to another entity.	T / E	M	H	C-14
T-15	A manager grants roles, or the last owner is removed.	E	M	H	C-20
T-16	A user attaches their entity under someone else's Household, injecting rows into the victim's roll-up reports.	T / S	L	M	C-19
T-17	Existence oracle: FK checks bypass RLS, so sequential IDs let an attacker probe for other tenants'	I	M	L	C-18

ID	Threat	STRIDE	L	I	Control(s)
	rows.				
T-18	SQL injection in `poly_app` sets `app.user_id` to anyone. The GUC is asserted by the app, not authenticated by the database.	S / I / E	L	H	C-44 prevents it; RLS does not (R-2)
T-19	Cross-tenant metadata read through user-defined entity types. This is by design (kickoff §2.4).	I	H	L	R-9; the C-14 view exposes the label only
T-20	Membership-table policies recurse through the authz functions, or FORCE RLS blinds those functions, so policies fail open or error.	E / D	M	H	C-15 (functions owned by `poly_authz`)

3.3 Ledger integrity — tampering & repudiation (`ledger`, `securities`, `posting`)

ID	Threat	STRIDE	L	I	Control(s)
T-21	A posted entry is edited or deleted through the app role, an ad-hoc SQL session, or `TRUNCATE`. `TRUNCATE` fires no row triggers and ignores RLS.	T / R	M	H	C-21, C-10 (no TRUNCATE grant)
T-22	An unbalanced entry is committed.	T	L	H	C-22
T-23	Forged attribution: `actor_id` is set to another user.	R / S	M	M	C-23
T-24	The operator superuser disables triggers (`ALTER TABLE … DISABLE TRIGGER`, `session_replication_role = replica`) and edits history undetectably. The same applies to editing a restored backup.	T / R	L	H	C-24 (proposed), R-1
T-25	A manager's `auto_post` rule posts attacker-shaped entries without review.	T	L	M	C-17 (manager+), C-23 (`actor =`

ID	Threat	STRIDE	L	I	Control(s)
					`rule:<id>`), C-53

3.4 TB-4 / TB-5 / TB-11 — Import, adapters & credentials

ID	Threat	STRIDE	L	I	Control(s)
T-30	A DB dump or backup leaks aggregator tokens, which are standing bearer access to real bank data.	I	M	H	C-30, C-72, C-73
T-31	Code execution or SSRF in the internet-facing `web` process opens every stored credential.	I / E	L	H	C-30 (`web` holds the public key only), C-31
T-32	Sealed-blob swap: someone with DB write copies the victim's sealed token into their own connection row, and the victim's bank data flows into the attacker's entity.	T / I	L	H	C-30 (AAD binds the blob to connection, entity and provider)
T-33	A token or URL leaks into logs, errors, API responses or the stored payload.	I	H	H	C-31, C-33, C-34, C-75
T-34	Unlink deletes the local row but leaves the token live at the provider.	I	M	M	C-32
T-35	A malicious CSV or OFX: XXE, billion-laughs, oversize input, spreadsheet-formula payloads.	T / D / I	M	M	C-40, C-41
T-36	Full account numbers stored in raw payloads widen the blast radius of any read leak.	I	H	M	C-34; payload readable by manager+ only (§4.2.1)

ID	Threat	STRIDE	L	I	Control(s)
T-37	A malicious attachment (a PDF/HTML polyglot, a forged content type, a header-injecting file name) runs in our origin, or attachments fill the disk.	T / E / D	M	M	C-46

3.5 Worker & jobs (`apps/worker`, `packages/refdata`)

ID	Threat	STRIDE	L	I	Control(s)
T-40	A job with blanket cross-tenant write: one bug corrupts every tenant.	E / T	M	H	C-60 (every tenant write has a user principal)
T-41	A job keeps importing after the linking user loses their grant or deletes their account.	E / I	M	M	C-60 (`principal_invalid`), C-52
T-42	An unauthenticated caller triggers the Vercel Cron internal route.	E / D	M	M	C-61
T-43	Tampered or garbage reference data (prices, CPI) skews valuations.	T	L	M	TLS, source priority, C-45

3.6 TB-9 — Export, account deletion, purge & audit

ID	Threat	STRIDE	L	I	Control(s)
T-50	Purge abuse: an owner who is not the sole owner, `poly_app` setting the purge GUC directly, a stale export, or writes made after the export.	T / E	L	H	C-50
T-51	The purge path deletes nothing and reports success. This happens under FORCE RLS with no DELETE policy.	T	H if built per	H	C-50 (`poly_purge` role-targeted DELETE

ID	Threat	STRIDE	L	I	Control(s)
			8963efb		policies; a zero-rows-remain assertion)
T-52	The purge erases its own evidence.	R	L	M	C-53
T-53	Export files linger on disk or in temp storage.	I	M	H	C-51
T-54	PRD §5's "rewrite the actor attribution" on account deletion would mean an UPDATE on append-only rows.	T	H	M	C-52 (tombstone the user row, never the entries)
T-55	An exported CSV runs a spreadsheet formula planted in a payee string.	T / E	M	M	C-41

3.7 TB-7 / TB-8 / TB-10 / TB-12 — Host, backups & supply chain

ID	Threat	STRIDE	L	I	Control(s)
T-60	Postgres or `web` is exposed publicly, because Docker-published ports bypass `ufw`.	I / E	M	H	C-70, C-71
T-61	A stolen disk, snapshot or backup exposes everything.	I	L	H	C-72, C-73
T-62	A box compromise also deletes or encrypts the off-box backups.	D / T	L	H	C-73 (append-only credential, immutable snapshots)
T-63	A compromised npm release (worm or typosquat) runs install scripts in CI or in the image.	E / I	M	H	C-80, C-82
T-64	A GitHub or Actions compromise pushes to prod through a deploy key.	E	L	H	C-81, C-83

ID	Threat	STRIDE	L	I	Control(s)
T-65	Code merged but not yet released runs on staging on the prod kernel, then escapes its container.	E	L	H	C-70, C-74, R-4
T-66	A secret is committed to the repo or baked into an image.	I	M	H	C-82, C-74

4. Authentication & Authorization

§4.3–§4.5 are the rulings on ARCH OQ-2, OQ-3 and OQ-4. OQ-1 is ruled in §6.3. §12 has the summary and the changes against 8963efb.

4.1 Authentication

- **Provider.** BetterAuth in our own Postgres, via the Drizzle adapter. Sign-in methods:
  - Email + password, hashed with scrypt (the BetterAuth default).
  - OAuth 2.0 with PKCE and a verified `state`, via Google, Apple or GitHub.
  - Passkeys.
  - Optional magic links sent through Resend.
- **Invite-only registration (C-01).** The first user on an empty instance is the bootstrap user. After that:
  - Self-signup is off. New users arrive only through a membership invitation from an entity owner.
  - OAuth sign-in for an unknown email with no pending invite is rejected.
  - An operator flag can re-open signup. It is off by default (Principal Q5).
- **MFA (C-03).** TOTP and passkeys, with single-use backup codes stored hashed.
  - *Proposed, pending Principal Q1:* MFA is required for any user holding owner or manager on any entity. OAuth-only users must enrol a passkey or TOTP to hold those roles, because we cannot verify the IdP's MFA.
  - 8963efb recommended MFA-optional, fearing lockout with no support desk. The lockout risk has a cheaper answer: an operator-side CLI (`polycarpic user reset-mfa <email>`). It runs only on the box and writes an `audit_event`. The operator is already the trust root (R-1).
- **Step-up.** These actions require a re-authentication within the last 5 minutes (password + MFA, or a passkey):
  - purge
  - entity export
  - granting a membership or changing a role
  - linking or unlinking a provider
  - disabling MFA
  - changing email
  - deleting the account
- **Web sessions (C-02).** Server-side BetterAuth records.

    - The cookie is `__Host-`-prefixed, `HttpOnly`, `Secure` and `SameSite=Lax`.
    - Sessions expire after 7 days idle or 30 days absolute.
    - All sessions are revoked on password change, MFA reset or account deletion.
    - Membership changes need no session revocation, because visibility is recomputed from `membership` in every transaction.
- **Native and MCP tokens (later).** Named, listable, individually revocable bearer tokens.
    - Stored hashed as Tier 1, and kept in the device Keychain.
    - Scoped to a *user*, never to an entity. A token grants what its user can do at the time of each request.
- **No in-app super-admin.** No application role reads across tenants. The operator's superuser is the only such principal, and it sits outside the app (R-1).
- **Service authentication.**
    - Each service uses its own Postgres role and password, delivered as files (C-74): `web` uses `poly_app`, `worker` uses `poly_worker`, migrate uses `poly_migrator`.
    - `web` and `worker` never talk over HTTP; they coordinate through pg-boss tables.
    - On Vercel, Cron calls an internal route authenticated by `Authorization: Bearer <CRON_SECRET>`, compared with `timingSafeEqual`. The route is compiled out of the `adapter-node` build (C-61).

4.2 Authorization model

RBAC over the membership join (`user`, `entity`, `role`), enforced by Postgres RLS. A user's rights come only from `membership` rows. They never come from a `user_id` column on a data table, and never from entity structure. Access is a grant; the hierarchy grants nothing.

`can(role, action)` in `packages/tenancy` exists to render a correct UI and to return a clean 403 early. It is *not* an enforcement point.

The design test: delete every `can()` call, and the system must still be safe. Any control that fails this test is app-layer-only authorization, and the security gate rejects it.

There are two exceptions, and both are enforced in database functions rather than `can()`: step-up recency (checked by the route, and required as an argument by the purge function) and sole-owner status.

4.2.1 Permission matrix (owner / manager / viewer)

Roles are the same for every entity type. The type drives tax routing and reporting, never authorization. The matrix is stored as data in `packages/tenancy`. Every row gets a pgTAP assertion that bypasses the app layer, plus an endpoint-level test (C-17).

Action	viewer	manager	owner	Enforced by
Read dashboard, balances, posted entries, lots, reports, drafts	✓	✓	✓	SELECT policy: `entity_id = any ((select app.visible_entities()))`
Read linked-account name and last-4	✓	✓	✓	Full numbers are never stored (C-34), so there is nothing to mask per role.
Read the raw provider payload (`raw_transaction.payload`)	×	✓	✓	Policy or column grant through a manager+ view. The payload is the least-curated, most PII-dense field we hold.
Read the entity's membership list	own row	✓	✓	SELECT policy on `membership`
Create, edit or delete a draft; confirm a draft (post); reverse an entry	×	✓	✓	Per-command policies with `WITH CHECK app.has_role(entity_id,'manager')`
Edit the chart of accounts; create or edit	×	✓	✓	As above. `auto_post` creation writes an `audit_event`.

Action	viewer	manager	owner	Enforced by
rules, including `auto_post`				
Upload CSV / OFX	×	✓	✓	Insert policy on `import_batch` and `raw_transaction`
Link a provider connection	×	✓ step-up	✓ step-up	Insert policy on `provider_connection`. The linking user becomes `linked_by_user_id`.
Unlink a connection	×	own links only	any, step-up	Policy compares `linked_by_user_id`. Revocation runs as a worker job (C-32).
Sidebar-IA actions (ARCH §2.4 @ 884748c). G6 in ARCH §3.2a relies on these rows.				
Refresh a provider token (relink)	×	✓ step-up, own links	✓ step-up	A relink stores a new sealed credential, so it is treated like a link (C-30). The policy compares `linked_by_user_id` for managers. Writes an `audit_event`.
Map a linked account to a custodial GL account	×	✓	✓	UPDATE policy on `linked_account`: manager+. The target account must belong to the same entity (composite FK). A mapping change writes an `audit_event`, because it redirects where bank data posts.
Sync now	×	✓	✓	Enqueues a worker job and never opens the credential in `web` (TB-4). Rate-limited per connection (minimum interval set in INFRA) against provider quota and cost.
Post, Split, Group, + New entry, Correct entry	×	✓	✓	G6: `WITH CHECK app.has_role(entity_id,'manager')` on `journal_entry` / `journal_line`, with the actor forced by trigger (C-23). Correct entry is a reversal plus a new

Action	viewer	manager	owner	Enforced by
				entry and never an edit (C-21). G1, G2, G4 and G5 are re-checked by the database at commit.
Rule auto-post opt-in (opting in counts as confirmation)	×	✓	✓	The opting-in user is recorded on the rule (`auto_post_by_user_id`) and is the principal for every auto-post, as in OQ-3. If that user drops below manager or is deleted, `auto_post` flips off and an `audit_event` is written, mirroring `principal_invalid`. Auto-posts pass G1–G6 like any post. Opt-in writes an `audit_event` and emails the owners.
Per-entity import schedule	×	✓	✓	UPDATE policy on the entity's import settings: manager+. The interval is bounded below by the same INFRA minimum as Sync now.
Set entity metadata (name, tax classification)	×	×	✓	UPDATE policy on `entity`: owner
Set a parent link	×	×	✓ on child *and* parent	C-19
Grant, revoke or change a membership; add or transfer an owner	×	×	✓ step-up	Owner-only policies on `membership`, plus a last-owner trigger (C-20)
Export the entity's complete books	×	×	✓ step-up	PRD §5 grants this to the owner. It is exfiltration-shaped, so it is audited (C-51). *Revised:* 8963efb allowed manager+.

Action	viewer	manager	owner	Enforced by
Purge the entity's books	×	×	sole owner, step-up	C-50 (§4.5)
Read the entity's `audit_event` rows	×	×	✓	SELECT policy: owner, or `actor_id = app.current_user_id()`
Clone or edit own report definitions; export own personal data; delete own account	self	self	self	Owner-user policies. Account deletion is refused while the user is sole owner of any entity (C-52).

pgTAP for the sidebar-IA rows (qa-engineer, C-17). Each row gets an allow test and a deny test, run as `poly_app` with the app layer bypassed:

- A viewer cannot run any of the actions: every INSERT or UPDATE is rejected by policy.
- A manager cannot relink or unlink another user's connection.
- A mapping to another entity's account fails the composite FK.
- An auto-post whose opting-in user has been demoted to viewer writes 0 rows, and its rule ends with `auto_post = false`.
- A "Correct entry" never UPDATEs the original; it leaves one reversal plus one new entry.
- Sync and schedule below the minimum interval are rejected.

Revocation semantics. Revoking a membership takes effect on the next transaction; there is no cached authorization. The user's sessions stay valid for their other entities. Data a client has already fetched cannot be recalled (R-11). The native milestone must purge its cache on any 401, 403 or empty scope.

4.3 RLS mechanism — ruling on ARCH §3.3 (OQ-2)

Ruling: confirmed, with amendments. These stand as designed: transaction-local GUC, STABLE SECURITY DEFINER visibility functions, ENABLE + FORCE on every tenant table, and default-deny.

Additions (a)–(d) are already in ARCH (from 8963efb) and remain in force:

- (a) The coverage probe is derived from the catalog.
- (b) No CREATE on `public`.
- (c) Per-command policies with `WITH CHECK` on every write; `FOR ALL` is banned.
- (d) The inversion canary stays.

Review found defects that (a)–(d) do not cover. Amendments (e)–(l) are new:

e. **FORCE RLS vs. the definer functions (latent defect, T-20).**
   - *The problem.* Under `FORCE`, the table *owner* is also subject to policies. ARCH has `visible_entities()` and `has_role()` owned by `poly_owner`, and they read `membership`, which is itself FORCEd.
   - *Why it breaks.* Once `membership`'s own SELECT policy calls `has_role()` (so that managers can list members), the chain function → `membership` policy → function makes Postgres raise "infinite recursion detected in policy". If the policy does not call it, the functions silently see only the rows the policy allows.
   - *Fix.* A `poly_authz` role (NOLOGIN, NOBYPASSRLS) with SELECT on `membership` and `entity_type` only, plus a role-targeted policy `CREATE POLICY authz_read ON membership FOR SELECT TO poly_authz USING (true)`. Both functions are owned by `poly_authz`. FORCE stays uniform and nothing gets BYPASSRLS. `poly_app` cannot use the policy, because it is not a member of `poly_authz`.

f. **The empty-string GUC (defect).**
   - *The problem.* 8963efb and ARCH say `current_setting('app.user_id', true)` returns NULL when unset. That is only true until the GUC is first touched in a session. After any transaction-local set, the placeholder reads back as `''`, and `''::uuid` raises. That is fail-closed, but noisy and fragile.
   - *Fix.* Every policy reads `app.current_user_id()`, which is `nullif(current_setting('app.user_id', true), '')::uuid`. The pool-reuse test runs its no-context probe on a connection that has already carried a context.

g. **`TRUNCATE` (defect).** It ignores RLS and fires no row-level DELETE triggers. *Fix:* a statement-level `BEFORE TRUNCATE` trigger that raises on every append-only table, and no login role holds

TRUNCATE.

h. **Pinned search_path.**
   - Definer functions use `SET search_path = ''`, and every name is schema-qualified.
   - EXECUTE is revoked from PUBLIC and granted explicitly.
   - Policies call these functions as `(select app.fn())`, so each is evaluated once per statement as an initplan.
   - A catalog allowlist names every SECURITY DEFINER function, and any function not on it fails the battery.

i. **Immutable tenancy columns.** `entity_id`, `actor_kind` and `actor_id` are left out of `poly_app`'s column-level UPDATE grants, so no row can change tenant or author. UPDATE policies carry both `USING` and `WITH CHECK`.

j. **Views.** Every view over tenant tables is created `WITH (security_invoker = true)`. Materialized views over tenant data are banned, because RLS cannot apply to them. The battery asserts both rules.

k. **Role hygiene.** `poly_app` and `poly_worker` are NOSUPERUSER, NOBYPASSRLS and NOCREATEROLE. They have no TRUNCATE, REFERENCES or TRIGGER grants and are members of no other role. The battery checks this with `pg_has_role` and `has_table_privilege`.

l. **Non-enumerable identifiers.** Primary keys for tenant-visible rows are UUIDs (v7 is acceptable). FK checks bypass RLS, so sequential IDs would make an existence oracle. ARCH §4.1's `/entities/42` becomes a UUID.

Pool safety. `set_config(…, true)` is safe under the in-process node-postgres pool and under transaction-mode pgbouncer, including Neon's pooler. The rules that keep it safe:

- Statement-mode pooling is banned, because it breaks transactions.
- The deploy checklist asserts the pooler mode on both targets (DevOps).
- `withTenant()` refuses to nest (AsyncLocalStorage guard).
- `withTenant()` accepts only a branded `SessionUserId`, which only `packages/auth` can construct.
- The raw pool is not exported, and the layer lint enforces that.
- Session-level `SET` is banned by lint.

What RLS buys, and what it does not.

- *It buys:* authorization bugs become empty results instead of leaks. That covers a missing `WHERE`, an IDOR, a forgotten table.

- *It does not buy:* any defence against SQL injection or code execution inside `poly_app`. The GUC is app-asserted, so anyone who can run SQL as `poly_app` can set it to anyone (R-2).
- *For that case:* prevention is C-44. The blast radius is capped by C-30 (`web` cannot open credentials) and C-10 (no DDL, no TRUNCATE).

4.4 System jobs — ruling on OQ-3

Ruling: confirmed as folded into ARCH. A job's tenant writes run under the linking user's membership. No tenant policy has a system bypass clause. The requirements already on POLY-64 stand:

- `provider_connection.linked_by_user_id` is NOT NULL.
- `withSystem(job, connection, fn)` sets `app.user_id` to that user and `app.actor = system:<job>`. Authority and attribution stay separate.
- If that user's membership drops below manager, or their account is deleted, the connection flips to `principal_invalid`. That state surfaces on the dashboard, writes an `audit_event` and notifies the owners. It never fails silently.
- A job never falls back to running context-free. If no principal resolves, it errors.

Additions:

- **Entity-bound jobs with no connection** (the tax-period freeze) run under a current owner of the entity, chosen deterministically as the earliest owner membership, with `actor = system:<job>`. Every tenant write therefore has a human principal.
- **Scheduling** needs a cross-tenant list of targets. That list comes from one allowlisted SECURITY DEFINER function, `app.job_targets(job)`, which returns only (`entity_id`, `principal_user_id`, `connection_id`). Only `poly_worker` can execute it.
- **Reference-data writes** (prices, CPI, tax tables) go through role-targeted policies (`TO poly_worker`). `poly_app` gets SELECT only. A role-targeted policy cannot be unlocked by any GUC that `poly_app` sets.
- **Cash-in-Transit aging** is computed on read by the exceptions report, not stored by a job. That removes a job that would need cross-tenant reads.
- **`poly_worker` tenant DML** is limited to the import path:
  - `import_batch`, `raw_transaction`, `reported_balance`
  - `draft`, `transfer_match`
  - status columns of `provider_connection`
  - `journal_entry` / `journal_line`, for `auto_post` only
  - `tax_period_freeze`, `audit_event`

4.5 The purge path — ruling on OQ-4 (revised)

Ruling: the purge function is owned by a dedicated NOLOGIN `poly_purge` role, with role-targeted DELETE policies. This reverses 8963efb, which had the function owned by `poly_owner` and rejected a separate role. The guard order ARCH folded in is kept, with two guards added.

- **Why 8963efb's shape fails (T-51).**
  - A definer function owned by `poly_owner` runs as the table owner, and under `FORCE ROW LEVEL SECURITY` the owner is subject to policies.
  - Append-only tables deliberately have no DELETE policy. So the function would delete *zero rows and report success*.
  - Adding a `poly_owner` DELETE policy would instead hand the purge capability to the migration owner.
- **Why the separate-role objection does not hold.** 8963efb said "a role needs a credential". A NOLOGIN role has no credential; it is only the owner identity for the definer function. The entry point is just as narrow: one function, with EXECUTE granted to `poly_app`. The only change is which role's privileges the function runs with.
- **Shape.**
  - `poly_purge` is NOLOGIN and NOBYPASSRLS, with DELETE on the purgeable tables only.
  - Each purgeable table gets `CREATE POLICY purge ON <t> FOR DELETE TO poly_purge USING (entity_id = nullif(current_setting('app.purge_entity', true), '')::uuid)`.
  - The append-only triggers allow DELETE only when `current_user = 'poly_purge'` *and* `OLD.entity_id` matches the GUC. `poly_app` can set the GUC but can never become `poly_purge`.
  - `app.purge_entity(entity_id, export_id, reauth_at)` is SECURITY DEFINER, owned by `poly_purge`, with `search_path = ''`.
- **Guards, in order, in one transaction:**
  1. Re-authentication within 5 minutes. The route checks it, and the function requires the timestamp as an argument.
  2. Sole-owner check against `app.current_user_id()`.
  3. An `export.completed` audit event for this entity, by this user, within 24 hours, carrying a SHA-256.
  4. *(New.)* No tenant write to the entity since the export's snapshot marker, so a manager's later postings cannot slip past the export. The entity row is locked `FOR UPDATE`.

5. An `entity.purged` `audit_event` is written before the first delete. It records the actor, entity id and name, the export event, per-table row counts and timestamps. `audit_event` has no FK to `entity` and no purge policy.
6. Every connection is revoked at the provider. The worker revoke job must have succeeded; a failed revocation blocks the purge and is never skipped.
7. Set `app.purge_entity` transaction-locally and delete in dependency order.
8. *(New.)* Assert zero rows remain for the entity in every purgeable table before commit. This is the fail-loud guard against T-51.

- **Not permitted:**
  - purge by a job;
  - purge through the API without interactive, typed confirmation of the entity name;
  - purge of a partial date range (that is an edit, and edits are reversals);
  - purge while a second owner exists.
- **Backups:** purged data persists until the backup retention window rolls (R-5). The purge warning says so.
- **Rejected alternatives:**
  - `ALTER TABLE … DISABLE TRIGGER` is DDL. It needs an ACCESS EXCLUSIVE lock and the table owner, and it affects every tenant.
  - Tombstoning is not a purge, so it breaks the PRD §5 promise.
  - BYPASSRLS is broader than needed.
- *Pending Principal Q2:* an optional delayed execution (24 h, cancellable, email to every member) as protection against a hijacked session.

5. Data Classification & Handling

User data falls into three sensitive tiers (from the Research brief), plus two non-tenant classes. Tiers 2 and 3 together are "sensitive personal financial data". Memo and payee text routinely leaks medical, income and relationship details, so it is not "just transactions".

Everything sits on an encrypted volume (C-72) and travels only over TLS. The table covers handling beyond that baseline. Assigning a tier to a new field is a security-gate question.

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
T1 Authentication secrets — Restricted	Password hashes, TOTP seeds, backup codes, session records, native/MCP bearer tokens, verification and reset tokens, IdP OAuth tokens	Passwords use scrypt. Backup codes and bearer tokens are hashed. Session-token hashing and `encryptOAuthTokens` in BetterAuth are unmeasured; POLY-62 confirms them, and R-12 applies otherwise. We request no offline OAuth scopes, because we need identity only.	Never logged. Never returned after creation. Never in a URL, except single-use reset and magic links that expire after 15 minutes.	Sessions: 7 days idle, 30 days absolute. Tokens: until used or expired. All deleted on account deletion.
T2 Standing bank-access credentials — Restricted (highest value)	SimpleFIN access URLs, Plaid access tokens, broker refresh tokens	HPKE-sealed, with AAD binding (§6.3, C-30). `poly_app` has no column SELECT (C-31).	Never sent to any client in any form, not even masked or partial. Never in DTOs, logs or the raw payload. Opened only in `worker`.	Until unlink. Revoked at the provider on unlink, account deletion or purge (C-32).
T3 Financi	Balances, transactions,	RLS on the membership join. Full account	Members only, per §4.2.1. Payloads	Ledger: indefinite

Tier	Data	At rest (beyond volume encryption)	Exposure rules	Retention
al data and PII — Confid ential	payees, memos, holdings, lots, entries, tax figures, entity names and classification, account last-4, raw payloads (redacted), exports, `audit_event`, user name and email	numbers are never stored (C-34). No field-level encryption: it would break sums, ranges and reports, for a threat that volume encryption plus RLS already covers.	are manager+ only. Logs carry opaque IDs only (C-33). Exports are streamed and never persisted (C-51). Emails are visible only to co-members who are owner or manager.	(append-only) until entity purge. Raw rows follow the ledger. `audit_event`: indefinite. On account deletion, the user row becomes a tombstone (C-52).
Opera tor secret s	Credential private keys, DB passwords, Plaid secret, Resend key, BLS key, OAuth client secrets, BetterAuth secret, backup key, `CRON_SECRET`	Files under `/run/secrets`, mode 0400. Never in the repo, an image or the database (§6.2).	Never logged. Prod and staging values differ. The backup key and the credential key are never stored together.	§6.4
T4 Public referen ce	Prices, CPI-U, tax tables, system report templates	—	No tenant column.	Indefinite

No outbound user data (a control). ARCH §1 says nothing about a user's books leaves the box, except through a provider call the user initiated or an owner-initiated export. Treat that sentence as a control. Any feature that would break it needs a security review and a PRD-level decision, not just a library import. Examples: crash reporting with payloads, hosted error tracking, telemetry that includes tenant data, or an LLM call that carries transaction text.

Stored, but never logged. T3 is stored in the clear under RLS, but scrubbed from logs. Logs get tailed over SSH, pasted into bug reports and shipped to aggregators, which makes them the likeliest escape route. That is why scrubbing is enforced by a CI canary (C-33) and not left to convention.

6. Secret Management

Two populations. Operator secrets: one set per instance, held by the person running the box. User credentials: one per connection, held sealed in the database. §6.3 resolves ARCH OQ-1.

6.1 Inventory

Secret	Lives in	Mounted into	Rotation
Credential keyring (X25519 private keys, by `key_id`)	`/etc/polycarpic/secrets/credential_keyring`, plus an offline copy in the operator's password manager	`worker` only. Public keys go to `web` as non-secret config.	§6.4
Postgres role passwords (`poly_app`, `poly_worker`, `poly_migrator`)	Host secrets directory	Only the container that uses that role	On compromise or a change of operator
BetterAuth secret	Host secrets directory	`web`	On compromise. Rotating it invalidates every session, which is the intended effect in an incident.
OAuth IdP client secrets, Resend key	Host secrets directory	`web`	At the provider, on suspicion
Plaid client secret, broker OAuth clients, price-provider key, BLS key	Host secrets directory	`worker`, plus `web` for the Plaid token exchange	At the provider, on suspicion
Backup encryption key and store credential	The public half (recipient) on the box; the private half offline (§6.6)	Backup job	On a change of operator. Keep the old key until its last backup expires.

Secret	Lives in	Mounted into	Rotation
`CRON_SECRET` (Vercel only)	Vercel env var	Internal cron route	On compromise
CI deploy credential or registry token	GitHub *environment* secret, with required reviewers	Actions runner	On compromise. Revocable independently of the operator's SSH key.
Aggregator tokens and URLs (T2)	Postgres, sealed (§6.3)	Opened only in `worker`	Driven by the provider. Re-sealed on key rotation.

6.2 Operator secret handling

- **Files, not environment variables.**
  - Secrets live on the host in `/etc/polycarpic/secrets/` (root:root, files 0400, directory 0700).
  - They are mounted as compose `secrets:` at `/run/secrets/`, and apps read them through the `*_FILE` convention.
  - Why: environment variables leak through `docker inspect`, `/proc/<pid>/environ`, child processes and error reporters that dump `process.env`.
  - *Revised:* 8963efb and ARCH §9 specify a root-only env file.
- **Per-container scoping.** Each container gets only the secrets it needs. `web` never gets the credential private key or the backup key. Staging gets its own values and no credential private key, because it uses only the fake provider.
- **Nothing in images.** No secret is ever baked into an image or passed as a build arg. `.env.example` lists names with a comment each, never values.
- **Fail closed at startup.** Outside development, the app refuses to start if a required secret is missing, or if a secret-looking env var is set in production.
- **Read once.** Secrets are read at startup. There is no runtime secret-fetch path and no admin endpoint that reveals configuration.
- **Vercel.** The secondary target uses Vercel's encrypted env vars (R-3).

6.3 Credential sealing — ruling on OQ-1 (revised)

Ruling: app-layer *asymmetric* sealing in `packages/secrets`. `web` holds only public keys; the private keyring is mounted into `worker` only.

This replaces 8963efb's symmetric envelope scheme, which ARCH §3.1 and TB-4 folded in: an AES-256-GCM DEK wrapped by a master key mounted into the app. The seam is unchanged: `seal()` / `open()` with a `key_id`. Rejecting pgcrypto and external KMS still stands (§6.5).

- **Why revise.**
  - With a symmetric master key, whoever seals can also open. The sealing process is `web`, because it receives the SimpleFIN setup token or the Plaid public token.
  - `web` is also the internet-facing process, and the most likely place for RCE or SSRF. So one `web` compromise would yield every bank token (T-31).
  - Asymmetric sealing removes that path at about the same implementation cost.
- **Algorithm.** HPKE (RFC 9180) base mode: DHKEM(X25519, HKDF-SHA256), HKDF-SHA256, AES-256-GCM. The library (e.g. `@hpke/core`, or an equivalent over `node:crypto`) is pinned in the `packages/secrets` design file, with SecEng sign-off. No home-rolled construction.
- **Binding.** The HPKE `aad` is the canonical encoding of (`connection_id`, `entity_id`, `provider`, `key_id`, `alg_version`). A blob copied to another row fails to open (T-32).
- **Stored columns.** `enc` (the encapsulated key), `ciphertext`, `key_id`, `alg_version`. These replace ARCH §3.1's "ciphertext + nonce + wrapped DEK + key_id".
- **Flows.**
  - *Link* (in `web`): receive the setup or public token, exchange it with the provider, `seal()` the result under the active public key, insert the row, drop the plaintext.
  - *Poll* (in `worker`): `open()` inside one call frame, for one provider call. The plaintext is never cached and never held in a module-level variable.
  - *Unlink*: `web` enqueues a revoke job. `worker` opens the credential, calls `adapter.revoke()`, then deletes (C-32).
- **Database enforcement.** `poly_app` has column INSERT and UPDATE on the sealed columns, but no SELECT. `poly_worker` has SELECT. The columns never appear in DTOs or `openapi.json` (C-31).
- **Errors.** Provider errors are re-thrown with the credential stripped. A raw provider exception never reaches a response or a log without passing through the scrubber.
- **Import boundary.** A lint boundary rule forbids any package except `packages/secrets` from importing the crypto module. Only `apps/worker` may import `open()`.

6.4 Key rotation for an operator with no KMS

1. Run `polycarpic secrets rotate` in `worker`. It generates key *n+1*, adds it to the keyring, and publishes its public key as the active sealing key.
2. It re-seals every row with `key_id = n`: open with *n*, seal with *n+1*. The step is idempotent and resumable.
3. It asserts that zero rows remain under key *n*, removes *n*, and writes an `audit_event`.
4. When to rotate: on any suspicion that the worker, the box, the operator's SSH key, or a backup together with the keyring was compromised, and whenever the operator changes. Annual rotation is optional hygiene. If the keyring *and* the database may both be exposed, rotation alone is not enough: revoke every token at the provider and relink (RB-1).

A lost keyring means relinking, not data loss. Every connection shows "relink required". The ledger is not encrypted under this key. The runbook says this plainly, so that nobody stores the key next to the backup "to be safe". The keyring is never part of the backup set.

6.5 Why not pgcrypto, and why not a secrets manager

- **pgcrypto (rejected).** The key and the plaintext would pass through SQL. That puts them in `pg_stat_activity`, potentially in statement logs, and in the same process as the data. A stolen dump must yield inert ciphertext. With the key co-located, it does not.
- **Vault or a cloud KMS (rejected for v1).** On a single VPS, a vault's unseal material sits on the same box. So it adds a service for a novice operator to run without adding real separation. It would also break the near-zero-spend guardrail and add an availability dependency. The `key_id` seam leaves room for a KMS-backed keyring later; revisit if R-6 ever triggers.
- **The credential-free path stays the recommended default.** The CSV/OFX adapter stores no credential. An instance that never links an aggregator holds zero T2 data. The operator runbook presents linking as a convenience trade-off.

6.6 Backup key custody — owned by DevOps (ARCH OQ-9)

Requirements on INFRA:

- Backups are encrypted on the box before they leave it.
- The key can be recovered without the VPS.
- The key is distinct from the credential keyring, and the two are never stored together.

Recommended shape:

- An asymmetric recipient (age-style), so the box holds only the public half.
- A write-only or append-only store credential, or immutable snapshots.

With that shape, a fully compromised box can neither read nor delete its own historical backups. The acceptance test is a restore onto a clean machine. RPO and RTO numbers are DevOps's call.

7. Controls Catalog

Each control maps to threats in §3. No application code exists yet, so Status names the milestone where the control must land. A feature that touches a control cannot pass the Validate security gate without it. Control IDs are new in this rewrite; 8963efb's C-numbers are retired.

ID	Control, location and test	Threats	Status
C-01	Invite-only registration after the bootstrap user. OAuth sign-in for an unknown email is rejected unless an invite is pending. **Lives at:** `packages/auth` hooks **Tested by:** Integration: a 2nd signup is rejected; OAuth for an unknown email is rejected; an invite accept works.	T-01	POLY-M1
C-02	Session cookie is `__Host-`, `HttpOnly`, `Secure`, `SameSite=Lax`. 7 days idle, 30 days absolute. All sessions revoked on a password or MFA change. **Lives at:** `packages/auth` config **Tested by:** Playwright `Set-Cookie` assertions; config unit test.	T-04	POLY-M1
C-03	MFA (TOTP, passkeys) per §4.1. Step-up re-auth within 5 minutes on the listed actions. Operator MFA-reset CLI, audited. **Lives at:** `packages/auth`; the `requireRecentAuth()` guard; worker CLI **Tested by:** Integration: every step-up action returns 403 without a recent re-auth; the CLI writes an `audit_event`.	T-02, T-08	POLY-M1 (MFA), then with each gated feature
C-04	BetterAuth `rateLimit` with database storage (e.g. sign-in 5/min per IP and per email). Generic auth error messages. Body-size limits on every route. **Lives at:** `packages/auth`; `hooks.server.ts` **Tested by:** Integration: the 6th attempt returns 429; responses for known and unknown emails are identical.	T-02, T-07, T-09	POLY-M1
C-05	OAuth uses PKCE and `state`. Accounts link only when the IdP asserts a verified email *and* there is either an existing session or a matching pending invite. **Lives at:** `packages/auth` **Tested by:** Integration: a mocked IdP sending `email_verified=false` does not link.	T-03	POLY-M1

ID	Control, location and test	Threats	Status
C-06	`kit.csrf.checkOrigin = true`, never disabled. Mutating `/api/v1` routes accept JSON only. SameSite cookies. **Lives at:** `svelte.config.js`; `/api/v1` wrapper **Tested by:** Integration: a cross-origin form POST is rejected; `text/plain` returns 415.	T-05	POLY-M1
C-07	CSP through `kit.csp`: nonce or hash, no `unsafe-inline` script, `frame-ancestors 'none'`, `object-src 'none'`. HSTS for 1 year at Caddy. `nosniff`. `Referrer-Policy: strict-origin-when-cross-origin`. **Lives at:** `svelte.config.js`; Caddyfile **Tested by:** Header assertions in the Playwright smoke run.	T-04, T-06	POLY-M1
C-10	Role separation and hygiene per §4.3(k), including `poly_authz` and `poly_purge`. No TRUNCATE, REFERENCES or TRIGGER grants to login roles. No CREATE on `public`. **Lives at:** `packages/db` role migrations **Tested by:** pgTAP catalog assertions.	T-13, T-21	POLY-M1
C-11	`withTenant()`: `set_config('app.user_id', …, true)` as the first statement; branded `SessionUserId`; nesting guard; raw pool not exported; session-level `SET` banned by lint; pooler-mode assertion in the deploy checklist. **Lives at:** `packages/db`; INFRA checklist **Tested by:** pgTAP pool-reuse test (tenant A, then tenant B on the same backend, sees 0 of A's rows); Vitest: a nested call throws.	T-11	POLY-M1
C-12	Fail-closed context: `app.current_user_id() = nullif(current_setting('app.user_id', true), '')::uuid`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: on a backend that already carried a context, a query with no context returns 0 rows on every tenant table and raises no error.	T-11	POLY-M1

ID	Control, location and test	Threats	Status
C-13	ENABLE + FORCE RLS and at least one policy on every table with a tenant column. The probe is catalog-derived, with an inversion canary. The global-table allowlist is a security-gate decision. **Lives at:** `packages/db` battery **Tested by:** pgTAP in CI; the build fails if any table is uncovered.	T-10, T-12	POLY-M1
C-14	Per-command policies with `WITH CHECK`; no `FOR ALL`. Tenancy and actor columns are not updatable. No DELETE policies on append-only tables. The entity-type exception goes through a label-only `security_invoker` view. **Lives at:** `packages/db` schema **Tested by:** pgTAP: a viewer INSERT is denied; an insert with a forged `entity_id` is denied; an `UPDATE` of `entity_id` gets permission denied.	T-10, T-14, T-19	POLY-M1
C-15	Authz functions are owned by `poly_authz`, with a role-targeted policy on `membership`. Every definer function sets `search_path = ''` and has EXECUTE revoked from PUBLIC. SECURITY DEFINER functions are allowlisted. **Lives at:** `packages/db` **Tested by:** pgTAP: the set of `prosecdef` functions equals the allowlist; `proconfig` pins `search_path`; a manager can list members without recursion.	T-13, T-20	POLY-M1
C-16	Views are `security_invoker = true`. No materialized views over tenant data. **Lives at:** `packages/db` **Tested by:** pgTAP catalog check on `reloptions` and `relkind`.	T-13	POLY-M1
C-17	The §4.2.1 matrix is stored as data. QA generates an endpoint × role test matrix, and a route with no matrix row fails CI. **Lives at:** `packages/tenancy`; QA suite **Tested by:** Generated Vitest + pgTAP tests; coverage check.	T-10, T-25	POLY-M1, growing with each feature

ID	Control, location and test	Threats	Status
C-18	Tenant-visible primary keys are UUIDs. No sequential IDs in URLs or the API. **Lives at:** `packages/db` schema **Tested by:** Schema test: every tenant PK has type `uuid`.	T-17	POLY-M1
C-19	A parent link requires owner on both the child and the parent. Policies never consult the parent link. **Lives at:** `packages/tenancy`; `entity` policy **Tested by:** pgTAP: owner of the child only is denied; a member of the parent sees no child rows.	T-16	POLY-M1
C-20	Only owners change membership. A trigger keeps at least one owner (except during purge). Every change writes an `audit_event`. **Lives at:** `packages/tenancy`; trigger **Tested by:** pgTAP: a manager's grant is denied; removing the last owner raises.	T-15	POLY-M1
C-21	Append-only: row-level BEFORE UPDATE/DELETE triggers plus a statement-level BEFORE TRUNCATE trigger. The only exception is `current_user = 'poly_purge'` together with a matching `app.purge_entity`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: UPDATE, DELETE and TRUNCATE all raise for `poly_app`, `poly_worker` and `poly_owner`; `poly_app` with the GUC set still raises.	T-21, T-50	POLY-M2
C-22	Deferred balance constraint trigger (ARCH §3.2). **Lives at:** `packages/ledger` SQL **Tested by:** pgTAP + property-based tests.	T-22	POLY-M2
C-23	A BEFORE INSERT trigger *forces* the actor from the session context. `actor_kind = user` requires `actor_id = app.current_user_id()`. **Lives at:** `packages/db` SQL **Tested by:** pgTAP: a forged `actor_id` is overwritten or rejected.	T-23, T-25	POLY-M2

ID	Control, location and test	Threats	Status
C-24	*Proposed:* a per-entity hash chain, `entry_hash = sha256(prev_hash ‖ canonical entry)`, written by the insert trigger. A verifier job checks the chain, and every export carries the chain head. **Lives at:** `packages/ledger` **Tested by:** pgTAP: an edit made as superuser is detected.	T-24	Post-GA, pending Principal Q4
C-30	HPKE sealing with AAD binding. `web` seals, `worker` opens; keys are held in a keyring by `key_id` (§6.3). **Lives at:** `packages/secrets` **Tested by:** Unit: round trip works; a swapped AAD, tampered ciphertext or unknown `key_id` all fail closed. Integration: the `web` container has no keyring file.	T-30, T-31, T-32	Seam in POLY-M2; first real use in POLY-1.0 (POLY-77)
C-31	`poly_app` has no SELECT on the sealed columns. DTOs and the OpenAPI spec omit them. **Lives at:** `packages/db`; DTOs **Tested by:** pgTAP: permission denied. OpenAPI freshness diff.	T-31, T-33	POLY-M2
C-32	Unlink enqueues a worker revoke job. The row stays `revoke_pending` until the provider confirms, with retries and backoff; a failure is surfaced to the user. Every step writes an `audit_event`. **Lives at:** `packages/import`; worker **Tested by:** Integration: when the fake provider's revoke returns 500, the row is not deleted.	T-34	POLY-M2 (fake provider), POLY-1.0
C-33	Logs are safe by construction. Pino serializers log only declared fields, and `redact` covers known paths. Never logged: bodies, `Cookie`, `Authorization`, SQL parameters, memo, payee. **Lives at:** Shared logger package **Tested by: Canary test:** the integration run seeds canary token, URL, account-number and memo values. CI captures every log line and fails if any canary appears.	T-33	POLY-M1 (harness), extended with each adapter
C-34	Adapters redact full account numbers, PANs and credential-bearing fields from stored payloads, keeping last-4 only. Payloads are readable by manager+ only.	T-33, T-36	POLY-M2, POLY-1.0

ID	Control, location and test	Threats	Status
	Lives at: Each `packages/import-*` **Tested by:** Per-adapter fixture containing canary numbers; pgTAP: a viewer cannot read the payload.		
C-35	`polycarpic secrets rotate` (§6.4). **Lives at:** `packages/secrets` CLI **Tested by:** Integration: after rotation every row opens under the new key and none under the old.	T-30, T-31	POLY-1.0
C-40	Upload caps (e.g. 10 MB, 50k rows). OFX 2.x parsed with DTD and external entities disabled; OFX 1.x parsed with a non-XML parser. No eval. **Lives at:** File adapter **Tested by:** Unit: XXE and billion-laughs fixtures are rejected; an oversize file returns 413.	T-35, T-09	With the file adapter (POLY-77)
C-41	Svelte escaping. `{@html}` is lint-banned everywhere except one sanitizing component. CSV export neutralizes any cell starting with `= + - @`, tab or CR. **Lives at:** `packages/ui` lint; export module **Tested by:** Unit: a `=HYPERLINK(…)` cell exports inert. Playwright: a `<script>` payee renders as text.	T-06, T-55	POLY-M1 (lint), then with the export feature
C-42	The report formula engine parses formulas to an AST over an account allowlist. `eval` and `new Function` are banned. Node-count, depth and time caps apply. **Lives at:** `packages/reports` **Tested by:** Unit: pathological and malicious formulas are rejected.	T-09	POLY-M5
C-43	Zod validates every route and form action. Unknown keys are stripped; a mismatch returns 400. **Lives at:** `apps/web` wrapper **Tested by:** Unit tests on the wrapper.	T-10, T-14	POLY-M1

ID	Control, location and test	Threats	Status
C-44	Parameterized queries only. Lint bans `sql.raw()` and `sql.unsafe` with non-literal arguments. Raw SQL is allowed only in reviewed migrations. CodeQL `js/sql-injection` runs in CI. **Lives at:** ESLint; CI **Tested by:** Lint + CodeQL.	T-18	POLY-M1
C-45	Reference-data anomaly flag: a price move over 50% day-over-day, or an out-of-bounds CPI revision, is stored but flagged. **Lives at:** `packages/refdata` **Tested by:** Unit.	T-43	POLY-M4
C-46	Attachments (ARCH §3.5 @ fcffe6c): • Stored as `bytea` under RLS. The bytes live in `attachment_blob`, separate from the metadata. • Size limit of 10 MB. Type is sniffed server-side against an allowlist; the client-declared type is never trusted. • Served only as a download: `Content-Disposition: attachment` with an RFC 6266 sanitized `filename*`, plus `nosniff` and a CSP of `default-src 'none'; sandbox`. • Composite entity FKs on attachments and their links. • Insert-only, and included in the purge. • Per-entity storage quota and upload rate limit. • File contents are never parsed. **Lives at:** `packages/attachments` (per ARCH) **Tested by:** Integration: a spoofed type is rejected; a file name containing CR/LF is sanitized; response header assertions; a cross-entity link is denied; purge leaves 0 attachment rows.	T-37	With the attachments feature
C-50	Purge per §4.5: the `poly_purge` role, 8 guards checked in order, and a typed confirmation. **Lives at:** `app.purge_entity`; tenancy route **Tested by:** pgTAP: a non-sole owner is denied; a stale export is denied; a write after the export is denied; `poly_app` with the	T-50, T-51, T-08	The purge feature (POLY-1.0 scope)

ID	Control, location and test	Threats	Status
	GUC set is denied outside the function; a successful purge leaves 0 rows in every purgeable table plus 1 `audit_event`.		
C-51	Export is streamed, never persisted. If it has to run async, the file is encrypted in temp storage and expires after 24 h. Owner only, with step-up; audited with a SHA-256; rate-limited. **Lives at:** Export module **Tested by:** Integration: no file remains after download.	T-53	The export feature
C-52	Account deletion: the user row becomes a tombstone; ledger rows are never UPDATEd. BetterAuth `deleteUser` is overridden. Sessions, accounts, MFA and passkeys are deleted. Linked credentials are revoked, and the user's connections go `principal_invalid`. Deletion is refused while the user is sole owner of any entity. **Lives at:** `packages/auth` override; tenancy **Tested by:** Integration: entries remain and resolve to "deleted user"; a sole owner's deletion is refused.	T-41, T-54	POLY-1.0
C-53	`audit_event` is append-only, has no purge exception and no FK to `entity`. Readable by owners and by the acting user. **Lives at:** `packages/db` **Tested by:** pgTAP: the row survives a purge and cannot be updated.	T-52, T-25	POLY-M1 (the table), then events with each feature
C-60	Job principal per §4.4: `linked_by_user_id`, `principal_invalid`, no context-free fallback, `app.job_targets()`, role-targeted reference-data policies, restricted `poly_worker` grants. **Lives at:** `packages/db`; `apps/worker` **Tested by:** pgTAP: a job whose linker was revoked writes 0 rows and flips the connection's state; `poly_app` cannot write reference data.	T-40, T-41	POLY-M2 (POLY-64)
C-61	Vercel topology (OQ-13): two Vercel projects from the same repo.	T-42, T-31	POLY-M1 (split); keyring

ID	Control, location and test	Threats	Status
	• `web` exposes only user routes. It holds only the public keys and the `poly_app` DB credentials. • `jobs` exposes only cron job routes. Each route checks `CRON_SECRET` with `timingSafeEqual`. It holds the keyring and the `poly_worker` credentials. • Job routes are absent from the node build and from the `web` project. **Lives at:** `apps/web`, `apps/worker` (Vercel entry) **Tested by:** Build tests: • the route manifest of each project equals its allowlist; • the `web` build contains no keyring reference; • the node build returns 404 for job routes. Env check in CI: `web` has no keyring variable.		before POLY-77
C-70	Only Caddy publishes ports. Prod and staging use separate compose networks. No Docker socket mounts. Containers run non-root, `read_only`, with `cap_drop: [ALL]` and `no-new-privileges`. **Lives at:** `compose.yml` (DevOps) **Tested by:** Compose policy lint in CI; a post-deploy external port scan shows only 22, 80 and 443.	T-60, T-65	POLY-M1 (INFRA)
C-71	SSH is key-only, with no root login. Unattended security upgrades. The firewall is the Hetzner Cloud Firewall, which sits outside the host, so Docker cannot bypass it. **Lives at:** INFRA provisioning **Tested by:** Provisioning check script.	T-60	POLY-M1 (INFRA)
C-72	The Postgres volume and local backup staging sit on an encrypted (LUKS) volume. **Lives at:** INFRA **Tested by:** Runbook verification.	T-61	INFRA v1
C-73	Backups per §6.6: asymmetric recipient, a key separate from the keyring, an append-only or immutable store, and a monthly restore drill.	T-61, T-62	Specified in ARCH; lands in

ID	Control, location and test	Threats	Status
	Lives at: ARCH §7.2 @ 6b2f11b: nightly `pg_dump` plus WAL, each encrypted to an age recipient; Hetzner Storage Box with an append-only subaccount, or S3 Object Lock **Tested by:** A monthly restore drill, run *off the box* because the private key never touches the VPS; a test that the store credential cannot delete.		POLY-M1 (INFRA)
C-74	Secrets are files, scoped per container (§6.2). Staging has its own distinct values. The app refuses to start if a secret is missing. No secrets in images. **Lives at:** INFRA; config loader **Tested by:** Trivy secret scan; config-loader unit test.	T-66, T-31	POLY-M1
C-75	Postgres logging: `log_parameter_max_length = 0`, `log_parameter_max_length_on_error = 0`, `log_statement = 'ddl'`. **Lives at:** Postgres config **Tested by:** Config assertion in the battery.	T-33	POLY-M1
C-80–83	Supply chain (§8). **Lives at:** CI **Tested by:** §8	T-63, T-64, T-66	POLY-M1

8. CI/CD Security

- **C-80 Dependencies.**
  - Installs use `pnpm install --frozen-lockfile`.
  - pnpm lifecycle scripts are blocked, with an explicit `onlyBuiltDependencies` allowlist.
  - `minimumReleaseAge` is at least 3 days in both pnpm and Renovate, so a freshly compromised release has time to be yanked before we pick it up.
  - Renovate never auto-merges major versions.
  - `pnpm audit --audit-level high` gates the build.
- **C-81 Workflow hardening.**
  - Actions are pinned by commit SHA.
  - Workflows default to `permissions: contents: read`, and widen per job only.
  - No `pull_request_target` that runs untrusted code.
  - No secrets are exposed to fork PRs.
- **C-82 Scanning.**
  - gitleaks runs in CI and as a pre-commit hook.
  - CodeQL (JS/TS) runs on PRs.
  - Trivy scans images and fails the build on fixable critical or high findings and on embedded secrets.
  - Each release carries a CycloneDX SBOM.
  - Each release image gets an `actions/attest-build-provenance` attestation.
  - No DAST in v1 (R-13).
- **C-83 Deploy path. Pull-based: adopted in ARCH §8.1 @ 6b2f11b.** A systemd timer on the box pulls the image by digest and verifies its build attestation before migrating. The attestation must be pinned to this repository and the release workflow, e.g. `gh attestation verify --repo richmosko/polycarpic --signer-workflow …`. A prod tag is accepted only if its signature verifies against the allowed signers file. The box holds a read-only token scoped to this repository with an expiry, or no token at all if the image package is public. The superseded push-over-SSH conditions are kept for reference:
  - the key is prod-only;
  - `authorized_keys` locks it to a `command=` forced script, with `no-pty` and `no-port-forwarding`;
  - the key lives in a GitHub environment with required reviewers;
  - staging uses a separate key.

- **Signing.** `/merge-pr` signs release tags (SSH signing) before the first prod deploy, because a tag triggers deployment. Per-commit signing is not required (R-10).

9. Logging, Monitoring & Detection

- **Logged:** request ID, user UUID, entity UUID, route, action, outcome, latency, job name and run ID.
- **Never logged:** T1 or T2 values, request or response bodies, memo or payee text, amounts, emails, cookies, `Authorization`, SQL parameters. The C-33 canary test enforces this.
- **Storage and retention.** JSON logs stay on the box, rotated, for 30 days. Postgres logging is set per C-75. An optional OTel export must pass through the same serializers.
- **Operator alerts**, sent by email through Resend:
  - more than 20 sign-in failures in 10 minutes;
  - MFA disabled, or a new passkey added;
  - a membership grant;
  - a provider link or unlink;
  - an export or a purge;
  - 3 failures of a revoke job;
  - a connection going `principal_invalid`;
  - a backup failure, or no successful backup within the RPO;
  - an RLS battery failure on deploy (this also blocks the deploy);
  - more than 50 authz denials or empty scopes per user in 10 minutes.

  The thresholds are provisional and unmeasured; tune them after POLY-M1. *Revised:* 8963efb accepted no alerting at all (its R-2). Email on these events costs almost nothing.
- **User-facing notices.** Users get an email for a sign-in from a new device, a password or MFA change, and being granted or removed from an entity.
- **Audit.** `audit_event` (C-53) survives purge. The journal actor columns (C-23) are the ledger's own audit trail. C-24 is the proposed tamper-evidence control.

10. Incident Response

- **On call:** each instance's operator. For the Principal's instance, that is the Principal. Alerts arrive by email (§9).
- **Severity:**
  - **SEV1**: suspected T1 or T2 exposure, or keyring exposure; a confirmed cross-tenant read or write; code execution on the box; ledger tampering detected.
  - **SEV2**: T3 exposure limited to one user; no backup within the RPO; a revoke that cannot complete; a critical CVE with a known exploit in a deployed dependency.
  - **SEV3**: a vulnerability with no evidence of exploitation; a threshold breach that turns out benign.
- **Runbooks.** These cover the security content only; exact commands live in the INFRA runbook.
  - **RB-1: keyring or worker compromise (SEV1).**
    1. Stop `worker`.
    2. Revoke every token at its provider: Plaid `/item/remove`, SimpleFIN bridge revocation, broker OAuth revoke. If the database and the keyring may both have been exposed, treat every token as burned.
    3. Generate a new keyring.
    4. Relink every connection.
    5. Rotate the provider API secrets.
    6. Review `audit_event` and the logs for credential opens that happened outside import runs.
  - **RB-2: web or box compromise (SEV1).**
    1. Put Caddy into maintenance mode.
    2. Snapshot the volume for forensics.
    3. Rotate every operator secret, including the BetterAuth secret. This invalidates all sessions.
    4. If the worker or the keyring was reachable, run RB-1.
    5. Rebuild from a clean, attested image on a fresh host.
    6. Restore the last backup taken before the compromise, and verify it against C-24 if that control exists.
  - **RB-3: cross-tenant leak bug (SEV1).**
    1. Disable the affected route.
    2. Add the failing case to the RLS battery or the authz matrix *before* fixing it.

3. Identify affected users from request logs, by route and user ID.
4. Notify the affected members.

- **RB-4: lost keyring (SEV3).** Generate a new keyring. Every connection then shows "relink required". The ledger is unaffected.
- **RB-5: backup failure or restore (SEV2).** Follow INFRA. Restore to staging first, and run the balance and hash verifier before promoting.
- **RB-6: compromised dependency (SEV1 or SEV2).**
  1. Pin to the last known-good version.
  2. Rebuild and redeploy.
  3. Diff the SBOMs to check whether the bad version shipped in a release image.
  4. If it did, run RB-2.

- **Post-incident:** a blameless postmortem within 5 business days as a cairn issue comment or review file (location per `process/WORKFLOW.md`). Record decisions in process/DECISIONS.md. The regression test lands with the fix.

11. Open Risks (Accepted)

Each risk has a rationale, a compensating control, a re-evaluation date and an approver. "Pending" means a SecEng recommendation that the Principal has not yet ruled on. ARCH cites R-1 and R-4, and both keep their meaning; the other numbers are reassigned from 8963efb.

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
R-1	The operator's superuser, and root on the box, bypass RLS. They can also disable the append-only triggers (`ALTER TABLE … DISABLE TRIGGER` , `session_replication_role`) and read the keyring.	The operator owns the box, the volume and the keys. No database control can bind them (PRD §6). *Corrected:* 8963efb said the triggers still stop the superuser. They do not.	C-24 hash chain (proposed): silent edits become detectable, and the chain head goes into every export. Superuser access only through `docker compose exec` or an SSH tunnel. C-71.	On any decision to host for others	Rich Mosko approved the principle (PRD §6, 2026-09-27); the corrected text is pending
R-2	RLS does not stop SQL injection or code execution in `poly_app` , because the app itself asserts the tenant GUC.	Binding the GUC to a DB-verified session token would gain little: `poly_app` has to read the session table for auth anyway.	C-44 (parameterized SQL, lint, CodeQL); C-30 (`web` cannot open credentials); C-10 (no DDL or TRUNCATE).	2027-03-31	pending

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
R-3	On Vercel, both projects live in one Vercel account. Anyone with access to that account or team, or a leaked Vercel token with env-read scope, can read the `jobs` keyring. Vercel functions are also public HTTPS endpoints, so the `jobs` project is reachable from the internet. It is guarded only by `CRON_SECRET`, with no user input. *Narrowed by OQ-13:* the web/worker split is kept.	Vercel is a secondary fallback target. The residual risk is administrative (account-level), not a web-process compromise.	Separate projects and env scopes (C-61). Vercel account MFA and least-privilege team membership. Rotate `CRON_SECRET` on suspicion. If the two-project split proves infeasible, Vercel may run only the fake or CSV adapters, never a real aggregator credential. Hobby-plan cron runs at most once a day (measured, ARCH §7.6 @ e3b4297). On Vercel, provider-side revocation after an unlink can therefore lag up to about 24 h, and the token stays live at the provider until then. The unlink UI must say so.	Before POLY-77 ships on Vercel	pending
R-4	Staging shares the prod kernel and auto-deploys unreleased code from `main` (TB-10).	It saves a second box. Staging holds fake data only, so what is at risk is a pivot into prod, not staging's own data.	C-70 (separate compose networks, non-root, read-only containers, no Docker socket); C-74 (distinct secrets, no keyring on staging); staging sits behind Caddy basic auth or an IP allowlist (ARCH §7.1).	ARCH §7.2 @ 6b2f11b keeps staging co-located. Re-evaluate if staging	pending

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
				ever touches non-fake data, or the app is offered at larger scale.	
R-5	Purged entity data persists in backups until the retention window rolls.	Rewriting encrypted backups is disproportionate.	The purge warning says so. INFRA bounds retention.	INFRA v1	pending
R-6	The multi-tenant design leaves a hosted offering possible. Reversing the non-goal would bring in GLBA, state privacy law and possibly PCI, and would make R-1, R-2 and R-13 unacceptable.	Hosting for others is a standing non-goal (PRD §3).	This row makes any reversal a deliberate, costed decision that re-opens §1.	Only on a deliberate decision to host	Rich Mosko (PRD §3, 2026-09-27)
R-7	Viewers are not required to use MFA. Conditional on Q1: if the Principal makes MFA optional for	Viewers cannot mutate data, link credentials or grant access.	C-04 rate limiting; onboarding leads with OAuth; new-device email (§9).	GA	pending (Q1)

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	all roles, this row widens to cover everyone.				
R-8	Losing the keyring makes sealed credentials unrecoverable, and rotation is a manual operator step.	Automating rotation would mean storing the new key somewhere automation can reach. Unrecoverable is the right failure mode here, because recovery is simply relinking.	The rotate command is idempotent and resumable (C-35); the runbook states the failure mode plainly; the ledger is not encrypted under this key.	Annually	pending
R-9	User-defined entity types are readable across tenants, by anyone with a membership on an entity of that type (kickoff §2.4).	A viewer needs the type label to render the entity.	A narrow policy plus a `security_invoker` view expose the label and tax defaults only, never the creator ID (C-14). Users are told that custom type names are shared.	If types ever carry more than a label	pending
R-10	Commits are unsigned.	With per-agent worktree identities, commit signatures prove little. Branch protection already prevents unreviewed merges.	Branch protection (admins included, no force-push, required `cairn` check). Release tags are signed (§8).	First prod deploy	pending
R-11	Revoking a grant does not recall data a client	This is inherent to any client. The next request is denied.	The native milestone must purge its cache on 401, 403 or an empty	Native milestone	pending

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	already fetched: an open browser tab now, the native cache later.		scope. SecEng reviews that milestone.		
R-12	*Conditional, unmeasured.* If BetterAuth stores session tokens unhashed, anyone with DB read can hijack sessions.	This needs DB read access, which already exposes T3.	Short sessions (C-02); step-up for sensitive actions (C-03). POLY-62 measures this and closes or confirms the row.	POLY-62	pending
R-13	No DAST, WAF or penetration test.	The surface is small and sits behind Caddy, and no regime requires one. pgTAP asserts the high-value properties more precisely than a scanner would.	The RLS battery; SAST, dependency, secret and image scanning (§8); this threat model is reviewed at every milestone.	GA	pending
R-14	Cross-root read access (PT-3, tooling). `cairn serve` reads directories named in a committed `roots:` config (or `--repos`), which can be outside the repo. A malicious `roots:` entry in a PR could point a reviewer's local board at a sibling	The server is read-only and localhost-only, with no write path to a secondary root. The `/api/board` payload carries only parsed cairn frontmatter, so there is no exfiltration channel. A directory without a `config.yml` is skipped before	When more than one root loads, a startup stderr banner logs each resolved absolute root path. It is a human tripwire against a stale or mistyped `roots:` entry silently serving the wrong tracker. Re-evaluate if cairn ever gains a write path to non-primary roots.	2027-02-21	Rich Mosko

#	Risk	Why acceptable	Compensating control	Re-evaluate by	Approved by
	directory, e.g. `../../.ssh`.	anything is read. Anyone who can commit a `roots:` entry already has a stronger vector, editing `cairn.py` or `board.js` directly, so a runtime path-jail would be theater.			

12. Rulings on ARCH Open Questions

12.1 Final rulings

OQ	Ruling	vs. 8963efb (as folded in ARCH @ eaa825e)	Detail
OQ-1	App-layer **asymmetric** sealing: HPKE with X25519, HKDF-SHA256 and AES-256-GCM. The AAD binds each credential to its connection, entity and provider. `web` seals; only `worker` holds the private keyring and can open. pgcrypto and KMS are rejected. Rotation is a re-seal job.	**Revised.** Under the old design, the symmetric master key was reachable from `web`.	§6.3–§6.5; C-30, C-31, C-35
OQ-2	ARCH §3.3 is confirmed, and additions (a)–(d) stand. This review adds (e)–(l): • (e) authz functions owned by `poly_authz`; • (f) context read as `nullif(…,'')`; • (g) TRUNCATE triggers; • (h) `search_path = ''` plus a definer-function allowlist; • (i) tenancy columns not updatable; • (j) views are `security_invoker`, with no materialized views; • (k) role hygiene; • (l) UUID keys. The permission matrix is the same for every entity type.	**Extended.** Export becomes owner-only, per PRD §5. Payload access stays manager+.	§4.2.1, §4.3; C-10 to C-20
OQ-3	Confirmed: the linking user is the job principal, `principal_invalid` exists, and there is no system clause. Additions: • entity-bound jobs run as the earliest owner; • a new `app.job_targets()` function; • reference-data policies are role-targeted; • Cash-in-Transit aging is computed on read.	**Extended.** The POLY-64 scope comment stays valid.	§4.4; C-60

OQ	Ruling	vs. 8963efb (as folded in ARCH @ eaa825e)	Detail
OQ-4	The purge function is SECURITY DEFINER, owned by a **NOLOGIN** `poly_purge` role. It uses role-targeted DELETE policies, and the append-only trigger makes an exception only when `current_user = 'poly_purge'`. ARCH's guard order is kept, plus two new guards: no writes since the export, and an assertion that zero rows remain.	**Revised.** A purge owned by `poly_owner` deletes zero rows under FORCE RLS (T-51).	§4.5; C-21, C-50

12.2 ARCH changes to re-fold (architect)

#	ARCH §	Change
A-1	§3.3	`visible_entities()` and `has_role()` are owned by `poly_authz`, with a role-targeted SELECT policy on `membership`. Add `poly_authz` and `poly_purge` to the role list.
A-2	§3.4	The purge function is owned by `poly_purge`. Add the role-targeted DELETE policies and key the trigger exception on `current_user`. Add guards 4 and 8. Replace the text "a separate purge role … rejected".
A-3	§3.3	Use the `nullif` form of `app.current_user_id()`. Correct the claim "unset returns NULL". The pool-reuse test must run on a backend that has already been used.
A-4	§3.2	Add BEFORE TRUNCATE statement triggers. No login role holds TRUNCATE.
A-5	§3.2	The actor is forced by trigger, not merely defaulted. `actor_kind = user` requires `actor_id = app.current_user_id()`.
A-6	§3.1, §4.1	Tenant-visible primary keys are UUIDs. Replace `/entities/42`.

#	ARCH §	Change
A-7	§3.1 `provider_connection`; §2.2 `packages/secrets`; §7 TB-4; §9	The sealed columns become `enc`, `ciphertext`, `key_id` and `alg_version`. `seal()` lives in `web`; `open()` lives only in `worker`. TB-4 becomes the web ↔ worker boundary. Revocation on unlink runs as a worker job. Operator secrets are files, not an env file.
A-8	§3.1 user; PRD §5 wording	On account deletion, the `user` row becomes a tombstone and entries are never UPDATEd. `actor_id` FKs are `ON DELETE NO ACTION`. Override BetterAuth's `deleteUser`.
A-9	§2.3 RawTxn	Adapters redact full account numbers, PANs and credential fields from the stored payload.
A-10	§3.3	Views are `security_invoker`, with no materialized views over tenant data. Tenancy and actor columns are not updatable.
A-11	§3.1 entity	A parent link requires owner on both the child and the parent.
A-12	§3.3 system jobs	Entity-bound jobs run as the earliest owner. Add `app.job_targets()`. Cash-in-Transit aging is computed on read.
A-13	§6.2 / POLY-76	Add spike acceptance criterion (e): FORCE RLS, a role-targeted `pgPolicy` (`to:`), and a definer function owned by a non-owner NOLOGIN role must round-trip through `generate` and apply as `poly_migrator`.
A-14	§3.1, §5 (*gap*)	Attachments appear in PRD §5 export and purge but have no storage design. They need: a location; size and type limits; serving with `Content-Disposition: attachment` and `nosniff`; and inclusion in both purge and export.
A-15	§7 TB list	Add TB-11 (upload ↔ parser) and TB-12 (supply chain ↔ prod).

12.3 Asks on other owners

- **devops-engineer:**
  - §6.2: secrets as files, scoped per container; the keyring goes to `worker` only.
  - §6.6: an asymmetric backup key and an append-only backup store.
  - C-70/C-71: only Caddy publishes ports; the firewall lives in the cloud provider, not on the host.
  - C-75: Postgres logging settings.
  - C-83: a pull-based deploy is preferred.
  - Assert the connection-pooler mode on both targets.
- **qa-engineer:**
  - Every row of §4.2.1 becomes a pgTAP assertion.
  - The battery adds: pool reuse on a dirty backend (C-12); WITH CHECK forgery; catalog coverage plus canary; the definer allowlist; `security_invoker`; TRUNCATE; purge negatives and the zero-rows-remain check (C-50).
  - The log canary (C-33) must land before the first real adapter.
- **Principal (via team-lead).** Five questions, each with SecEng's recommendation:
  - Q1: Require MFA for the owner and manager roles? *Recommend yes.* The operator reset CLI covers lockout recovery.
  - Q2: Run a purge only after a cancellable 24 h delay, emailing every member? *Recommend yes.*
  - Q3: Approve R-2 to R-5 and R-7 to R-13, and re-confirm R-1's corrected text? *Recommend approving all.*
  - Q4: Build the ledger hash chain (C-24) in v1, or after GA? *Recommend after GA.*
  - Q5: Make registration invite-only after the bootstrap user, with a reopen flag that is off by default? *Recommend yes.*
